

BIOS 731 Homework 4: Bayesian Inference and Cluster Computing

```
source(here("source", "gibbs_sampler_fn.R"))
source(here("source", "variational_inference_fn.R"))
```

Problem 0: Reproducibility

The GitHub repository containing this work can be found at

https://github.com/ryanjtaylor97/bios731_hw4_taylor.

Problem 1: Gibbs sampler

We have the data-generating model:

$$\begin{aligned}\mu &\sim N(0, \sigma^2) \\ c_i &\sim \text{Categorical}(1/K, \dots, 1/K) \\ y_i | c_i, \mu &\sim N(c_i^\top \mu, 1).\end{aligned}$$

Then first, for the full conditional distribution $p(c_i | y_i, \mu)$, note that c_i can take on one of K different values: $(1, 0, \dots, 0)$; $(0, 1, 0, \dots, 0)$; $\dots$; $(0, \dots, 0, 1)$. Each of these possibilities has probability $p(c_{ik} = 1) = \binom{1}{1! * (0!)^{K-1}} * (\frac{1}{K})^1 * (\frac{1}{K})^{K-1} = \frac{1}{K}$. In addition, when $c_{ik} = 1$, $c_i^\top \mu = \mu_k$. Therefore, we can write

$$\begin{aligned}
p(c_{ik} = 1 | y_i, \mu) &\propto p(c_{ik} = 1) * p(y_i | c_{ik} = 1, \mu) \\
&\propto \exp\left(-\frac{1}{2}(y_i - \mu_k)^2\right) * \frac{1}{K} \\
\Rightarrow [c_i | y_i, \mu] &\sim \text{Categorical}\left(\frac{\phi(y_i; \mu_k, 1)}{\sum_{k=1}^K \phi(y_i; \mu_k, 1)}\right),
\end{aligned}$$

where $\phi(\cdot; \mu, \sigma^2)$ is the pdf of a univariate Normal distribution with mean μ and variance σ^2 .

Second, since we assume all y_i 's are independent of each other, we can derive an update all μ_k parameters at the same time using the entire data vector y and the entire $n \times K$ matrix of group assignments c .

$$\begin{aligned}
p(\mu | y, c) &\propto p(y | c, \mu) * p(\mu) \\
&\propto \exp\left(-\frac{1}{2}(y - c^\top \mu)^\top (y - c^\top \mu)\right) * \exp\left(-\frac{1}{2}\mu^\top \mu\right) \\
&\propto \exp\left\{-\frac{1}{2}\left[\mu^\top \left(c^\top c + \frac{1}{\sigma^2} \mathbf{I}_K\right) \mu - 2\mu^\top c^\top y\right]\right\} \\
&\propto \exp\left\{-\frac{1}{2}\left[\mu - \left(c^\top c + \frac{1}{\sigma^2} \mathbf{I}_K\right)^{-1} c^\top y\right]^\top \left(c^\top c + \frac{1}{\sigma^2} \mathbf{I}_K\right) \left[\mu - \left(c^\top c + \frac{1}{\sigma^2} \mathbf{I}_K\right)^{-1} c^\top y\right]\right\} \\
\Rightarrow [\mu | y, c] &\sim N\left(\left(c^\top c + \frac{1}{\sigma^2} \mathbf{I}_K\right)^{-1} c^\top y, \left(c^\top c + \frac{1}{\sigma^2} \mathbf{I}_K\right)^{-1}\right).
\end{aligned}$$

Since c is a matrix of indicator vectors, $c^\top c$ is a diagonal matrix, and therefore we can rewrite this as K univariate normal distributions so we do not need to invert a matrix:

$$c^\top c = \begin{bmatrix} n_1 & 0 & 0 \\ 0 & n_2 & 0 \\ 0 & 0 & n_3 \end{bmatrix},$$

where $n_k = \sum_{i=1}^n c_{ik}$. Therefore, we can write $[\mu_k | y, c] \sim N\left(\frac{\sum_i c_{ik} y_i}{n_k + 1/\sigma^2}, \frac{1}{n_k + 1/\sigma^2}\right)$.

These full conditionals are implemented in the Gibbs sampler in the file `source\gibbs_sampler_fn.R`.

Problem 2: Variational inference

We need to estimate the “latent” parameters μ, c . In order to estimate these, we’ll also define the variational parameters $m = \mathbb{E}_q(\mu)$, $s^2 = \text{Var}_q(\mu)$, and $\psi = \mathbb{E}_q(c)$, so that $\psi_{ik} = P(c_{ik} = 1)$. Then by a mean-field approximation, we can write

$$\begin{aligned}
ELBO(q(\mu, c)) &= E_q[\log p(y, \mu, c)] - E_q[\log q(\mu, c)] \\
&= E_q[\log p(y|\mu, c) + \log p(\mu) + \log p(c)] - E_q[\log q(\mu) + \log q(c)] \\
&= \sum_{i=1}^n \{E_q[\log p(y_i|c_i, \mu)] + E_q[\log p(c_i)]\} + \sum_{k=1}^K E_q[\log p(\mu_k)] - \\
&\quad \sum_{i=1}^n E_q[\log q(c_i)] - \sum_{k=1}^K E_q[\log q(\mu_k)].
\end{aligned}$$

Then by the formula we proved in class, the CAVI update for c_i is

$$\begin{aligned}
q^*(c_i) &\propto \exp\left\{\log p(c_i) + \mathbb{E}_{q(\mu)}[\log p(x_i|c_i, \mu)]\right\} \\
&\propto \exp\left\{-\log(K) + \mathbb{E}_{q(\mu)}\left[\sum_{k=1}^K c_{ik} * \left(-\frac{1}{2}(x_i - \mu_k)^2\right)\right]\right\} \\
&\propto \exp\left\{\sum_{k=1}^K c_{ik} * \mathbb{E}_{q(\mu)}\left(-\frac{1}{2}(x_i - \mu_k)^2\right)\right\} \\
&\propto \exp\left\{\sum_{k=1}^K c_{ik} * \left[E(\mu_k)x_i - \frac{1}{2}E(\mu_k^2)\right]\right\} \\
&= \exp\left\{\sum_{k=1}^K c_{ik} * \left[m_k x_i - \frac{1}{2}(s_k^2 + m_k^2)\right]\right\} \\
\Rightarrow \psi_{ik}^* &\propto \exp\left[m_k x_i - \frac{1}{2}(s_k^2 + m_k^2)\right]
\end{aligned}$$

Likewise, the CAVI update for μ_k is

$$\begin{aligned}
q^*(\mu_k) &\propto \exp\left\{\log p(\mu_k) + \mathbb{E}_{q(c)}[\log p(x_i|c_i, \mu)]\right\} \\
&\propto \exp\left\{-\frac{\mu_k^2}{2\sigma^2} + \sum_{i=1}^n \mathbb{E}_{q(c)}[c_{ik} * \log p(x_i|\mu_k)]\right\} \\
&\propto \exp\left\{-\frac{\mu_k^2}{2\sigma^2} + \sum_{i=1}^n \left[\psi_{ik} * \left(-\frac{1}{2} * (x_i - \mu_k)^2\right)\right]\right\} \\
&\propto \exp\left\{\mu_k^2 \left[\frac{-1}{2\sigma^2} + \sum_{i=1}^n \frac{-\psi_{ik}}{2}\right] + \mu_k \left[\sum_{i=1}^n \psi_{ik} x_i\right]\right\} \\
&\propto \exp\left\{\frac{-1}{2} \left(\frac{1}{\sigma^2} + \sum_{i=1}^n \psi_{ik}\right) * \left[\mu_k^2 + 2\mu_k \left(\frac{1}{\sigma^2} + \sum_{i=1}^n \psi_{ik}\right)^{-1} \left(\sum_{i=1}^n \psi_{ik} x_i\right) + \left(\frac{1}{\sigma^2} + \sum_{i=1}^n \psi_{ik}\right)^{-2} \left(\sum_{i=1}^n \psi_{ik} x_i^2\right)\right]\right\} \\
\Rightarrow q^*(\mu_k) &= N\left(\frac{\sum_{i=1}^n \psi_{ik} x_i}{\frac{1}{\sigma^2} + \sum_{i=1}^n \psi_{ik}}, \frac{1}{\frac{1}{\sigma^2} + \sum_{i=1}^n \psi_{ik}}\right) \\
&= N(m_k, s_k^2).
\end{aligned}$$

Finally, we can rewrite the ELBO using these updates:

$$\begin{aligned}
ELBO(q) &= \sum_{i=1}^n \{E_q[\log p(y_i|c_i, \mu)] + E_q[\log p(c_i)]\} + \sum_{k=1}^K E_q[\log p(\mu_k)] - \\
&\quad \sum_{i=1}^n E_q[\log q(c_i)] - \sum_{k=1}^K E_q[\log q(\mu_k)] \\
&= \sum_{i=1}^n \{E_{q(c)}[E_{q(\mu)}[-\frac{1}{2}(y_i - \mu_k)^2|c] - \log K] + \sum_{k=1}^K E_q\left[-\frac{\mu_k}{2\sigma^2}\right] - \\
&\quad \sum_{i=1}^n E_q\left[\log\left(\prod_{k=1}^K \psi_{ik}^{c_{ik}}\right)\right] - \sum_{k=1}^K E_q\left[-\frac{1}{2}\log(2\pi s_k^2) - \frac{1}{2s_k^2}(\mu_k - m_k)^2\right] + \text{constant} \\
&= \sum_{i=1}^n \sum_{k=1}^K -\frac{\psi_{ik}}{2} [(y_i - m_k)^2 + s_k^2] - \sum_{k=1}^K \frac{1}{2\sigma^2} (m_k^2 + s_k^2) - \sum_{i=1}^n \sum_{k=1}^K \psi_{ik} \log(\psi_{ik}) + \sum_{k=1}^K \frac{1}{2} \log(2\pi s_k^2) + \text{constant}
\end{aligned}$$

We use this definition of the ELBO to monitor convergence in the CAVI algorithm implemented in the file `source\variational_inference_fn.R`.

Problem 3: Fit to data

After loading the Old Faithful data, we divide the waiting times by their empirical standard deviation so that the data we input into our model will have unit variance.

```
source(here("source", "apply_to_data.R"))
```

We then run three separate Gibbs sampler chains with assumed $\sigma^2 = 100, 10, 1$. These three assumptions all lead to very similar results, suggesting that these solutions are not a local optimum.

```
kable(of_gibbs_100_summ$mean_summary %>%  
  mutate(across(matches("group"), ~ . * scale_factor)) %>%  
  filter(statistic != "variance"),  
  caption = "Means: Variance 100") %>%  
kable_classic(full_width = F)
```

Table 1: Means: Variance 100

statistic	group_1	group_2
mean	68.15275	73.63400
2.5%	64.14516	70.40909
97.5%	71.46391	77.56997

```
kable(of_gibbs_10_summ$mean_summary %>%  
  mutate(across(matches("group"), ~ . * scale_factor)) %>%  
  filter(statistic != "variance"),  
  caption = "Means: Variance 10") %>%  
kable_classic(full_width = F)
```

Table 2: Means: Variance 10

statistic	group_1	group_2
mean	68.16592	73.50901
2.5%	64.08645	70.35672
97.5%	71.39628	77.46409

```
kable(of_gibbs_1_summ$mean_summary %>%
  mutate(across(matches("group"), ~ . * scale_factor)) %>%
  filter(statistic != "variance"),
  caption = "Means: Variance 1") %>%
kable_classic(full_width = F)
```

Table 3: Means: Variance 1

statistic	group_1	group_2
mean	67.68597	73.02506
2.5%	63.56182	69.81220
97.5%	70.91705	77.01671

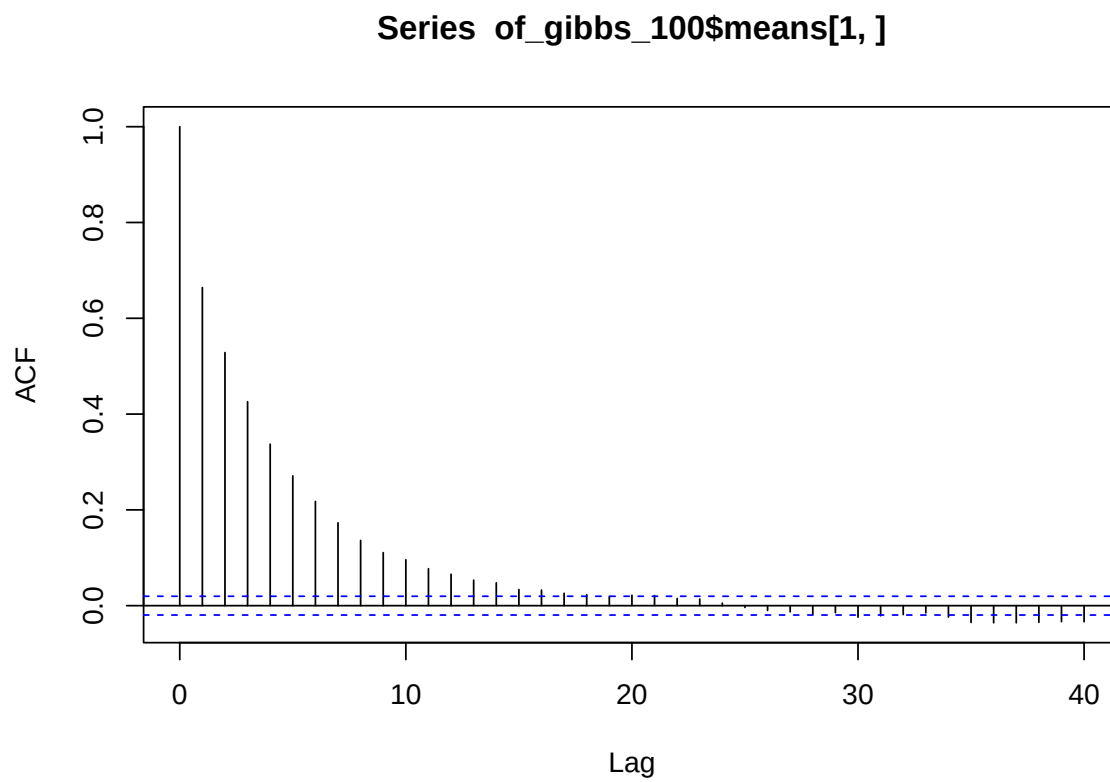
Effective sample size (compared to 10,000 total iterations) and auto-correlation plots suggest that these models are mixing fairly well.

```
ess_tbl <- tibble(
  sigma_2 = c(100, 100, 10, 10, 1, 1),
  group_mean = rep(c(1,2), 3),
  ess = c(
    effectiveSize(of_gibbs_100$means[1,]),
    effectiveSize(of_gibbs_100$means[2,]),
    effectiveSize(of_gibbs_10$means[1,]),
    effectiveSize(of_gibbs_10$means[2,]),
    effectiveSize(of_gibbs_1$means[1,]),
    effectiveSize(of_gibbs_1$means[2,])
  )
)

kable(ess_tbl) %>% kable_classic(full_width = F)
```

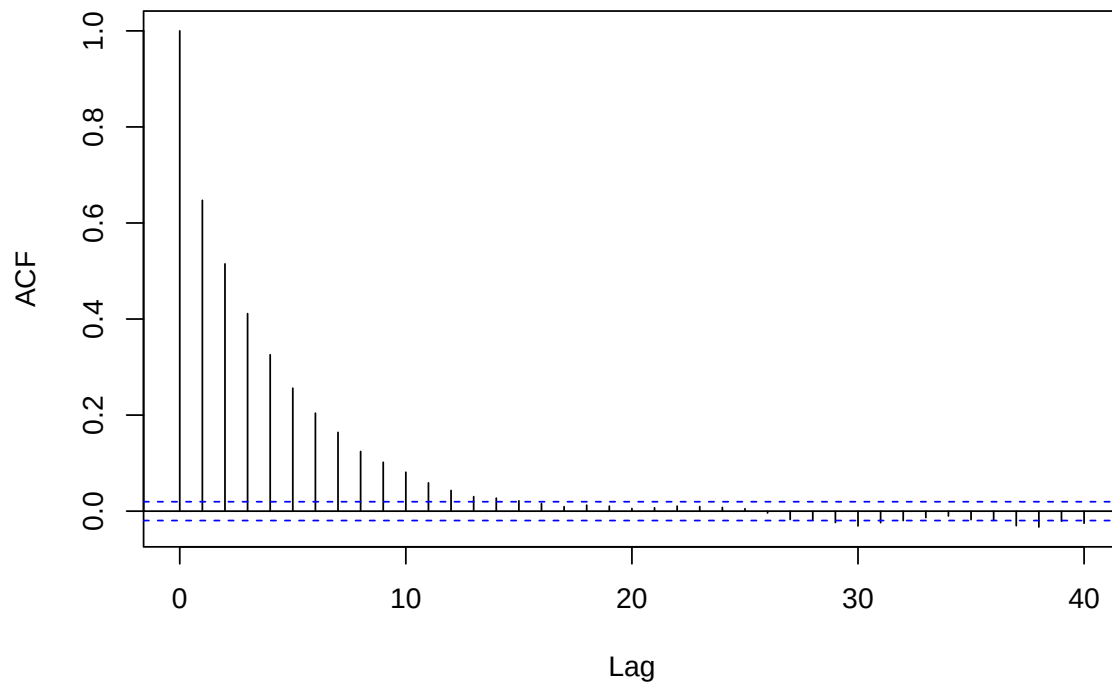
sigma_2	group_mean	ess
100	1	1338.840
100	2	1398.235
10	1	1343.587
10	2	1339.987
1	1	1388.678
1	2	1449.766

```
acf(of_gibbs_100$means[1,])
```



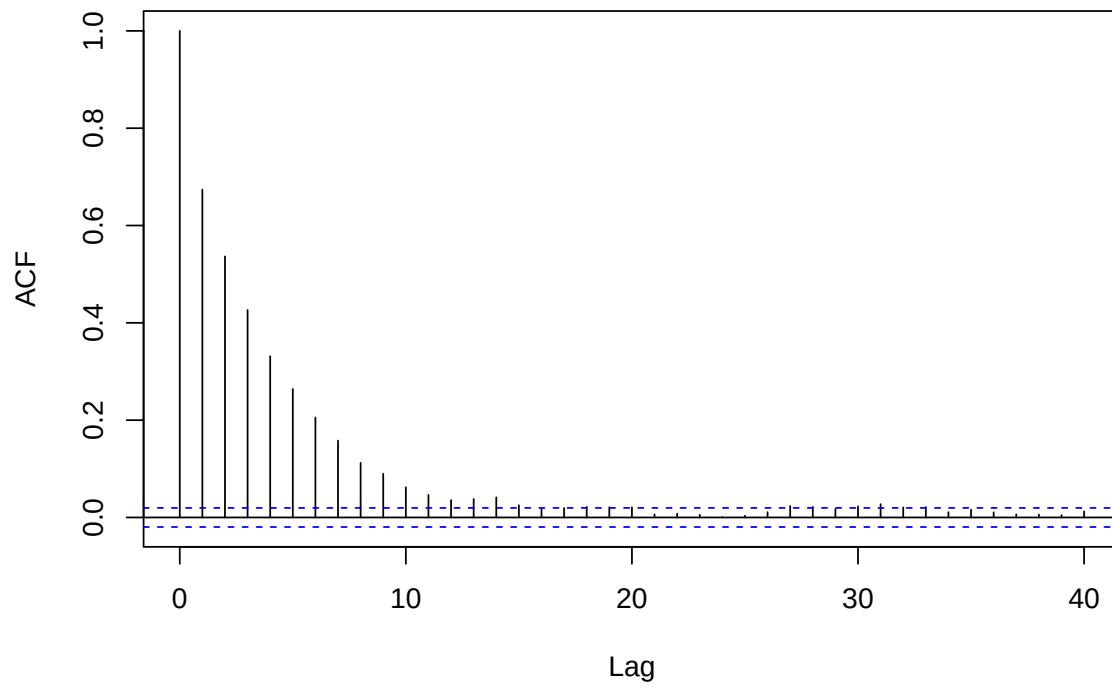
```
acf(of_gibbs_100$means[2,])
```

Series of_gibbs_100\$means[2,]



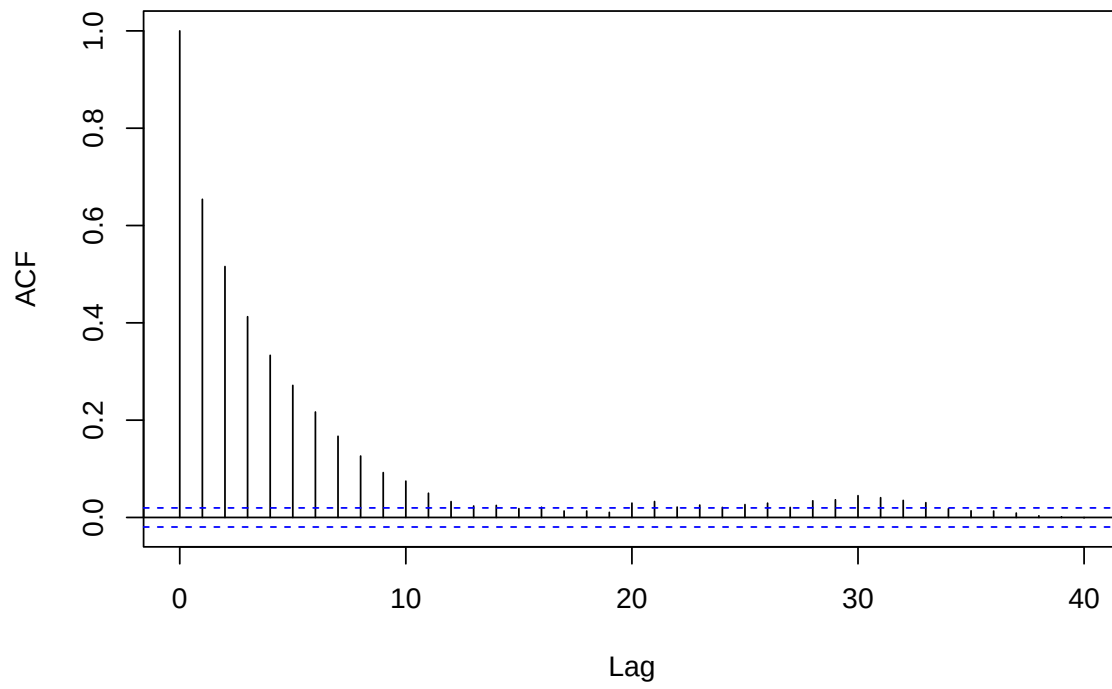
```
acf(of_gibbs_10$means[1,])
```


Series of_gibbs_10\$means[1,]



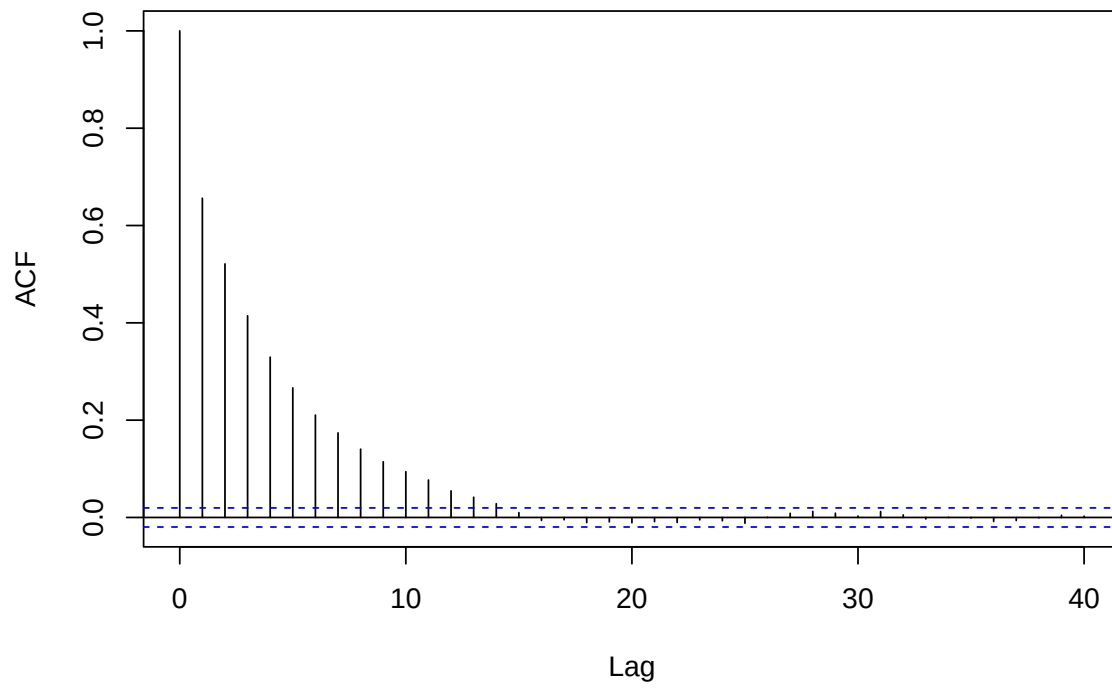
```
acf(of_gibbs_10$means[2,])
```

Series of_gibbs_10\$means[2,]

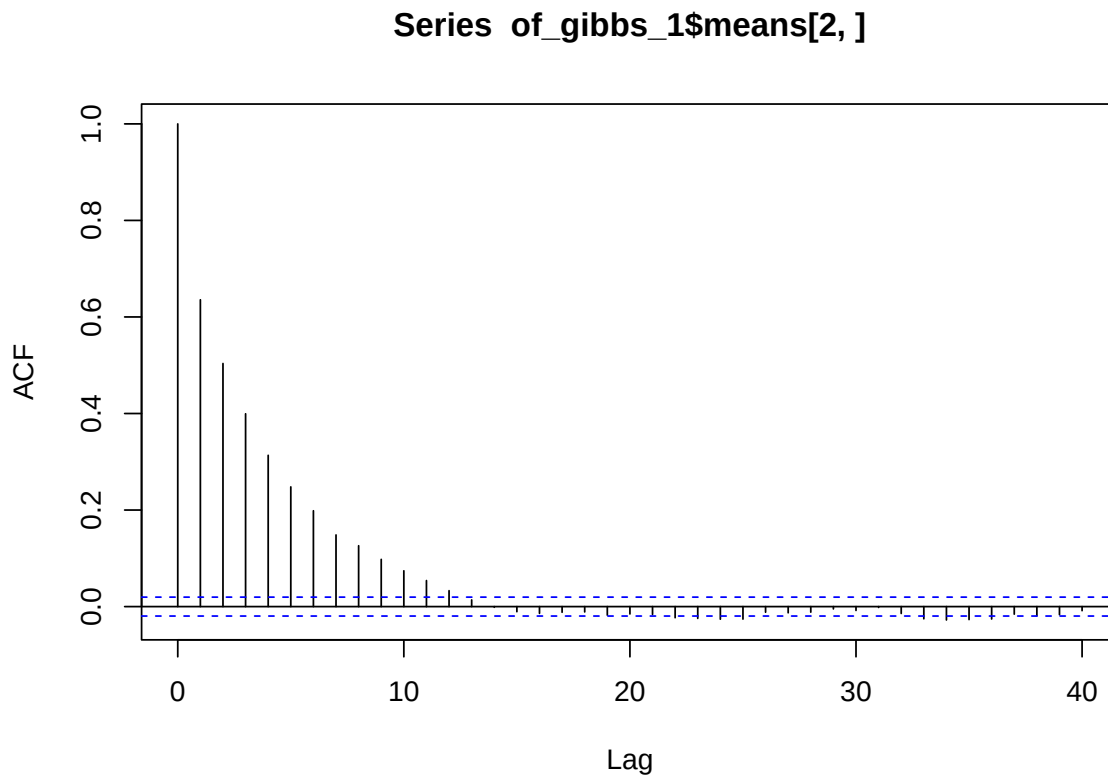


```
acf(of_gibbs_1$means[1,])
```

Series of_gibbs_1\$means[1,]



```
acf(of_gibbs_1$means[2,])
```



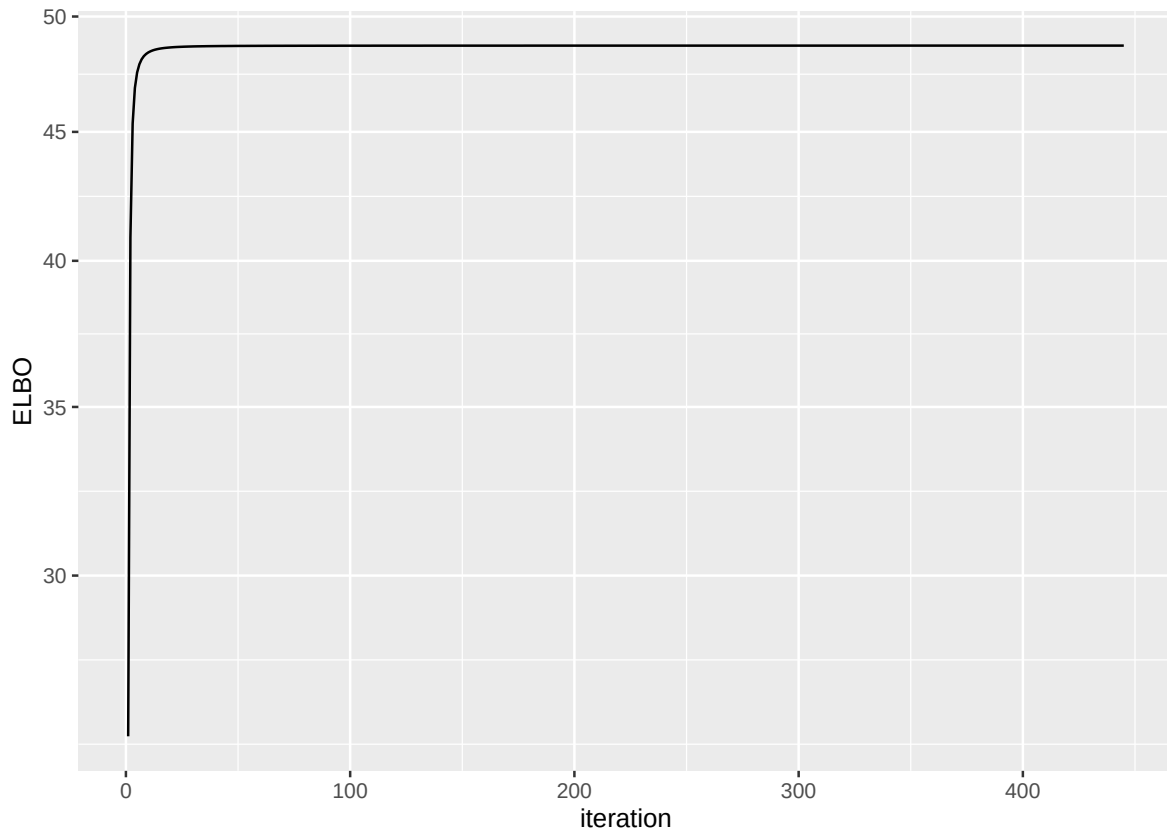
To compare to these models, we fit a CAVI algorithm to the same data. In order to avoid plateaus in the ELBO before reaching a true maximum, we label the model as converged if the ELBO changes by less than the tolerance level for 10 iterations.

```
cavi_means <- of_cavi$means * scale_factor
cavi_means
```

```
      [,1]
[1,] 70.68926
[2,] 71.09443
```

The group mean estimates from the CAVI algorithm are much closer together. It is likely that the CAVI algorithm settled into a local optimum instead of a global optimum. We tried changing the starting values for the means to address this, but CAVI estimates of the optimal group means ended up close to the overall mean of the data. A plot of the ELBO shows it has converged.

```
ggplot(data = enframe(of_cavi$elbo)) +
  geom_line(aes(x = name, y = value)) +
  scale_y_continuous(transform = "pseudo_log") +
  labs(x = "iteration", y = "ELBO")
```



On the other hand, the CAVI algorithm converged almost 40 times faster than the Gibbs sampler ran with 10,000 iterations.

```
time_tbl <- tibble(
  "model" = c("gibbs 100", "gibbs 10", "gibbs 1", "cavi"),
  time = c(last(of_gibbs_100$time$time),
            last(of_gibbs_10$time$time),
            last(of_gibbs_1$time$time),
            last(of_cavi$time$time))
)
```

```
kable(time_tbl) %>% kable_classic(full_width = F)
```

model	time
gibbs 100	4.044
gibbs 10	3.939
gibbs 1	3.965
cavi	0.088

```
mean(time_tbl$time[1:3]) / time_tbl$time[4]
```

```
[1] 45.25758
```

Problem 4: simulation study on the cluster

The last scenario did not finish running for us, and we were not able to produce plots of our results.

```
source(here("source", "process_sim_results.R"))
```

However, results from these simulations show that there is still likely a problem with the estimates from our CAVI algorithm. These estimates rarely cover the true group means, whereas Gibbs sampler-derived estimates cover the true mean in about 82-86% of simulations. Likewise, the bias for our Gibbs-sampler derived estimates is low, demonstrating that we are able to recover the true group means using this method. On the other hand, the bias of our CAVI-derived group means is high, and these estimates again appear to be driven by the overall mean instead of specific group means.

```
kable(bias_df) %>% kable_classic(full_width = F)
```

method	n	group	nsim	bias	var_bias	se_bias
CAVI	100	group_1	500	8.7363099	0.0010717	0.0327374
CAVI	100	group_2	500	3.7363099	0.0010717	0.0327374
CAVI	100	group_3	500	-1.2636901	0.0010717	0.0327374
CAVI	100	group_4	500	-11.2636901	0.0010717	0.0327374
CAVI	1000	group_1	500	8.7546265	0.0001128	0.0106193
CAVI	1000	group_2	500	3.7546265	0.0001128	0.0106193
CAVI	1000	group_3	500	-1.2453735	0.0001128	0.0106193

CAVI	1000	group_4	500	-11.2453735	0.0001128	0.0106193
Gibbs	100	group_1	500	0.0340776	0.0002049	0.0143137
Gibbs	100	group_2	500	0.1418405	0.0010292	0.0320806
Gibbs	100	group_3	500	0.3863775	0.0072143	0.0849372
Gibbs	100	group_4	500	-0.0030318	0.0000921	0.0095995
Gibbs	1000	group_1	500	0.0479975	0.0001455	0.0120639
Gibbs	1000	group_2	500	0.1396569	0.0009619	0.0310137
Gibbs	1000	group_3	500	0.3781851	0.0070651	0.0840540
Gibbs	1000	group_4	500	0.0040377	0.0000103	0.0032152

```
kable(coverage_df) %>% kable_classic(full_width = F)
```

method	n	group	coverage	nsim	var_coverage	se_coverage
CAVI	100	group_1	0.000	500	0.0000000	0.0000000
CAVI	100	group_2	0.000	500	0.0000000	0.0000000
CAVI	100	group_3	0.082	500	0.0023524	0.0485013
CAVI	100	group_4	0.000	500	0.0000000	0.0000000
CAVI	1000	group_1	0.000	500	0.0000000	0.0000000
CAVI	1000	group_2	0.000	500	0.0000000	0.0000000
CAVI	1000	group_3	0.000	500	0.0000000	0.0000000
CAVI	1000	group_4	0.000	500	0.0000000	0.0000000
Gibbs	100	group_1	0.834	500	0.0043264	0.0657752
Gibbs	100	group_2	0.824	500	0.0045320	0.0673201
Gibbs	100	group_3	0.826	500	0.0044914	0.0670177
Gibbs	100	group_4	0.858	500	0.0038074	0.0617039
Gibbs	1000	group_1	0.824	500	0.0045320	0.0673201
Gibbs	1000	group_2	0.822	500	0.0045724	0.0676193
Gibbs	1000	group_3	0.834	500	0.0043264	0.0657752
Gibbs	1000	group_4	0.860	500	0.0037625	0.0613392